# FitTrack AI Project Documentation

**SMARTBRIDGE - FitTrack AI Project Documentation**

Date: 21/09/2026

Team ID: SWTID-2026-4223

Team Size: 4

Team Leader: Sweatha P

Team Members:Lakshmi Narayanan R, Rithika P, Sandhiya K

Project Name: AI FitTrack

Maximum Marks: 5 Marks

## Project Description

AI FitTrack (FitSense) is a smart, AI-driven fitness tracking RESTful API backend engineered with Node.js, Express.js, MongoDB Atlas, and Google Gemini Generative AI (@google/genai). The system provides personalized AI-powered fitness insights, custom workout plan recommendations, secure user authentication (JWT + bcrypt), and comprehensive workout logging capabilities.

## Core Modules

1. User Authentication & Profile Management: Secure user registration, encrypted login via bcryptjs, JWT token verification, and profile data retrieval.

2. Workout Logging & Analytics CRUD: Complete management for logging exercises with details like workout name, category, duration, calories burned, and date filtering.

3. AI Workout Plan Generator: Generates custom workout regimes tailored to a user's age, fitness goals, and experience level using Google Gemini AI.

4. AI Fitness Insights Engine: Processes user workout histories to calculate total volume, average workout durations, and total calories burned, returning tailored performance summaries.

## Use-Case Scenarios

Scenario 1 (New User Onboarding & Registration): A new user signs up on AI FitTrack. The system encrypts their password using bcryptjs with 10 salt rounds and returns a signed JWT token for secure API calls.

Scenario 2 (Personalized AI Plan Generation): A user submits their age (20), target goal ("shoulder & chest growth"), and experience level ("beginner"). The API sends a prompt to Google Gemini AI (gemini-2.5-flash), returning a structured workout routine in under 3 seconds.

Scenario 3 (Workout Logging & Analytics Insight): An active athlete logs multiple running and strength sessions over a week. The user requests performance insights; the AI module evaluates their total duration and calorie stats, returning actionable feedback to optimize recovery.

## Technical Architecture & ER Diagram

The system utilizes a 3-tier REST API architectural pattern:

```
User / Client (Postman / Web Frontend)
        ↓ JSON over HTTP REST API / JWT
Node.js + Express.js API
        ├── Routes & Auth Middleware (JWT)
        └── Controllers & Gemini AI Service
            ├── MongoDB Atlas (Cloud Database)
            └── Google Gemini AI (@google/genai API)
```

## Entity Relationship (ER) Schema Description

User Entity: _id (ObjectId), name (String), email (String, Unique), password (Hashed String), createdAt (Timestamp).

Workout Entity: _id (ObjectId), user (Ref ObjectId -> User), workoutName (String), category (String), duration (Number in mins), caloriesBurned (Number), workoutDate (Date).

Relationship: 1-to-Many Relationship — One User can log multiple Workout records (User.id == Workout.user).

## Project Workflow & Milestones

### Milestone 1: Project Setup & Gemini AI Integration

Activity 1.1: Created Google Gemini API Key from Google AI Studio and configured .env file securely.

Activity 1.2: Evaluated Gemini models and selected gemini-2.5-flash for high-speed inference and structured text generation.

Activity 1.3: Structured project repository into modular MVC architecture (config, controllers, middleware, models, routes, services).

Activity 1.4: Installed core Node.js dependencies (express, mongoose, dotenv, @google/genai, jsonwebtoken, bcryptjs, cors).

### Milestone 2: Authentication & Database Models
Activity 2.1: Configured MongoDB Atlas cloud database connection in src/config/db.js.

Activity 2.2: Developed Mongoose schema for User with automatic bcryptjs password hashing pre-save hooks.

Activity 2.3: Created authController.js and authMiddleware.js for JWT Bearer token authentication.

### Milestone 3: Workout Management Core Logic
Activity 3.1: Built Workout schema with references to the authenticated user ID.

Activity 3.2: Developed workoutController.js containing CRUD operations: addWorkout, getWorkouts, updateWorkout, and deleteWorkout.

### Milestone 4: Gemini AI Service Integration
Activity 4.1: Programmed src/services/geminiService.js utilizing the official @google/genai SDK.

Activity 4.2: Developed /api/ai/workout-recommendation and /api/ai/fitness-insights endpoints in aiController.js.

### Milestone 5: Testing & Verification
Activity 5.1: Conducted API endpoint testing using Postman/Thunder Client.

Activity 5.2: Executed load and latency performance tests, verifying mean response latency under 200ms for core CRUD APIs.

## Core Implementation Code Snippets

### 1. Google Gemini AI Service (src/services/geminiService.js)
```
const { GoogleGenAI } = require('@google/genai');
const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
const modelName = process.env.GEMINI_MODEL || 'gemini-2.5-flash';
const generateWorkoutRecommendation = async (age, fitnessGoal, experience) => {
  const prompt = `Act as an expert fitness trainer. Provide a personalized workout
recommendation for a person with:
  - Age: ${age}
  - Fitness Goal: ${fitnessGoal}
  - Experience Level: ${experience}
  Keep the advice clear, actionable, structured, and under 200 words.`;
  const response = await ai.models.generateContent({
    model: modelName,
    contents: prompt,
  });
  return response.text;
```

```
};
module.exports = { generateWorkoutRecommendation };
```

### 2. User Model with Bcrypt Hashing (src/models/User.js)

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
}, { timestamps: true });
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};
module.exports = mongoose.model('User', userSchema);
```

## Conclusion & Future Enhancements

AI FitTrack provides a scalable RESTful API backend that combines workout tracking with AI generative recommendations. By leveraging MongoDB Atlas, Node.js, and Google Gemini AI (gemini-2.5-flash), the platform provides secure, high-speed fitness intelligence.

1. Integration with wearable devices (Apple HealthKit / Google Fit SDK).

2. Calorie and nutrition meal plan generation via Gemini Vision API.

3. Web/Mobile frontend interface built with React.js or Flutter.